



KTH Technology and Health

TENTAMEN

Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	CMEDT, TIDAA, TIELA, TIMEL -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	2024-10-21 8:00 - 13:00
Hjälpmedel:	IntelliJ finns på tentamenskontot Kort syntaxlista delas ut på tentamen. API-dokumentation finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt och hälften av poängen på B-delen samt att minst en av de två första programmeringsuppgifterna är helt korrekt lösta. B-delen rättas endast om minst hälften av poängen på A-delen uppnåtts. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. Alla lösningar skrivs i ett och samma projekt, med varje uppgift i ett separat paket, med namnen uppg9, uppg10, ... <i>När du skapar projektet i IntelliJ måste du kontrollera att projektet hamnar under enheten H, på ditt tentamenskonto.</i>

Mac-användare: För att komma åt måsvingar, hakparenteser och "|", använd "AltGr" + lämplig tangent, t.ex. "AltGr" + "{" (7:ans tangent).

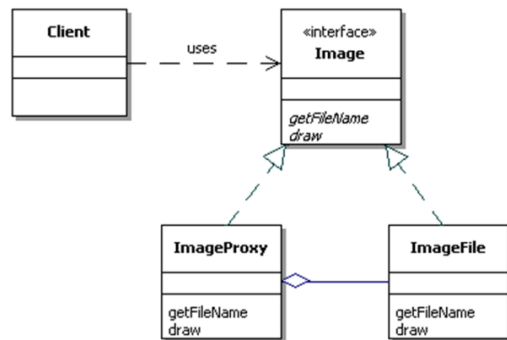
Del A, Teoriuppgifter

Svara, på separat papper, kort men koncist på frågorna. På fritextfrågor ska svaren motiveras. Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

I uppgift 1–4 ska du ange vilket eller vilka alternativ som är korrekta. Svaren ges på ett gemensamt blad.

1. Vilka är de två huvudsakliga syftena med inkapsling i objektorienterad kod?
 - A. Att förhindra att objektens tillstånd kan ändras, dvs. att objekten är "immutable".
 - B. Att dölja den interna implementationen i klassen och utåt, dvs. för andra klasser, endast exponera den nödvändiga funktionaliteten.
 - C. Att via en interface-referens dölja vilken verklig implementation av interfacet som klient-kod anropar.
 - D. Att säkerställa att objekten i alla lägen följer kontraktet för den aktuella typen (klassen).(2 p)
2. Vilka av nedanstående alternativ får ingå i en abstrakt klass i språket Java?
 - A. Abstrakta metoder
 - B. Metoder med implementation
 - C. Datamedlemmar (instansvariabler)
 - D. Konstruktörer(2 p)
3. Vilket av nedanstående påståenden om hög kohesion i objektorienterad kod är korrekt?
 - A. Hög kohesion minskar behovet av duplicering av kod, vilket gör det lättare att underhålla/uppdatera koden.
 - B. Hög kohesion syftar på att implementationen av en klass, och dess metoder, fokuserar på att abstrahera en enda roll eller funktionalitet i applikationen.
 - C. Hög kohesion medför att ändringar i den inre implementationen i en klass har minimal påverkan på andra klasser som är beroende av denna klass.
 - D. Hög kohesion medför sämre prestanda när koden exekveras.(2 p)
4. Vilket av följande påståenden är korrekt vad gäller skillnaden mellan vanlig aggregation och komposition i objektorienterad programmering?
 - A. Aggregation innebär att objekt är starkt beroende av varandra, medan komposition innebär att objekt är oberoende av varandra.
 - B. I aggregation kan de ingående objekten existera oberoende av varandra, medan i komposition kan inte delarna existera utan det överordnade objektet.
 - C. Aggregation använder arv för att skapa en stark relation, medan komposition använder gränssnitt för att upprätta samband.
 - D. I aggregation kan ett objekt endast vara en del i ett enda överordnat objekt, medan i komposition kan ett objekt tillhöra flera olika överordnade objekt.(2 p)

5. Klassdiagrammet nedan beskriver en variant av designmönstret Proxy. Vad innebär symbolen mellan
- a) ImageProxy och Image?
 - b) ImageProxy och ImageFile?



(2 p)

6. Anta att vi i en i en metod ska anropa en (annan) metod för att öppna och läsa från en fil. Vi vill att om ett undantag (exception) kastas ska detta inte hanteras i metoden, men metoden ska garantera att kopplingen till filen stängs, oavsett om undantag kastas eller inte. Skriv kod som visar hur metoden ska se ut vad gäller de kodblock som hör ihop med undantagshantering. Koden för hur filen öppnas och stängs behöver inte vara helt korrekt vad gäller klass- och metodnamn, men det är viktigt att referenser hanteras korrekt.

(2 p)

7. Vad innebär nyckelordet synchronized i Java och till vad används nyckelordet? Ge ett enkelt, men meningsfullt, exempel på användning av nyckelordet.

(2 p)

8. Beskriv, med pseudokod, designmönstret Singleton. Ange också i vilken/vilka situationer det är lämpligt att använda mönstret.

(2 p)

Del B, Programmeringsuppgifter

Lösningar lämnas i ett gemensamt projekt på tentamenskontot. För varje lösning ska det finnas ett paket med namnet "uppg9", "uppg10" et cetera. Till dessa uppgifter kan du i vissa fall ha hjälp av Javas API-dokumentation, som finns på P:/Java/api.

Alla klasser i denna del av tentamen ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha meningsfulla konstruktörer, som korrekt initierar alla datamedlemmar, access-metoder samt en omdefinierad version av toString.

9. Många former

A) För att representera ritbara och flyttbara figurer behöver vi ett interface, IShape, med de abstrakta metoderna

- move(int dx, int dy) och
- draw().

Skriv interfacet IShape samt en abstract klass, AShape, som implementerar IShape. AShape ska ha *privata* datamedlemmar som representerar en punkt och en färg. En punkt representeras av en klass, eller Record, Point med två heltal, x och y. Färgen representeras av en enum, Color, med lämpliga värden.

Klassen AShape ska ha nedanstående publika metoder.

- Två konstruktorer, en parameterlös som sätter lämpliga default-värden samt en som har argument för varje datamedlem.
- Accessmetoder för datamedlemmarna.
- En implementation av move som flyttar punkten önskat antal steg (dx, dy).

Skriv sedan nedanstående subklasser till AShape. Båda klasserna ska implementera metoden draw så att *komplett* information om objektet tillstånd skrivs ut på terminalen. Båda klasserna ska ha två konstruktorer, en parameterlös och en med argument för alla datamedlemmar, samt en omdefinition av draw och lämpliga accessmetoder.

- Line, som har ytterligare en punkt, änden på linjen. Metoden move ska omdefinieras så att båda punkterna förflyttas.
- Circle, som förutom ärvt data har en radie.

Skriv en main-metod där du skapar Line- och Circle-objekt, placerar dessa i en lista med IShape-referenser, itererar över listan och anropar metoderna move och draw.

B) Klassen Group ska implementera IShape och internt kunna lagra objekt av alla subtyper till IShape. Det ska finnas en metod add för att kunna lägga till IShape-objekt samt omdefinitioner av move och draw (som utför operationerna på alla objekt i gruppen).

Skriv en main-metod (slå ihop med A) där du skapar objekt av de olika subtyperna till IShape, inklusive Group. Skapa ytterligare ett objekt av typen Group och visa att man kan lägga grupper i grupper. Demonstrera anrop av move och draw på gruppen.

(4 + 2 p)

10. Icke ändringsbar text

A) Klassen Text ska representera texter som inte kan ändras. Texten representeras internt i en char-array. Klassen ska ha nedanstående metoder.

- Två konstruktorer, en som tar en char-array (kopiera tecknen till den interna arrayen) och en som tar ett String-objekt som argument. Om indata är null ska ett exception kastas.
- length, som returnerar aktuellt antal tecken.
- charAt(int index), som returnerar teckent på angiven position. Ett exception ska kastas om index är felaktigt.
- replace(char oldCh, char newCh), som ersätter alla förekomster av oldCh med newCh.
- concat(Text other), som skapar, och returnerar, ett nytt Text-objekt, med den sammanslagna texten.

Skriv klassen Text samt en main-metod som demonstrerar funktionaliteten.

B) Lägg till att klassen Text implementerar det generiska interfacet Comparable samt omdefinierar metoden Object::equals. Jämförelsen i compareTo(Text other) gör du genom att jämföra tecken för tecken; om du hittar två olika tecken returnerar du skillnaden mellan dessa, om alla tecken är lika (och textsträngarna är lika långa) returnerar du 0.

Det är lämpligt att använda compareTo när du implementerar equals.

Skriv en main-metod där du demonstrerar funktionaliteten (gemensam kod för A och B går bra).

(4 + 2 p)

11. For-each med Strategy

Skriv en (begränsad) implementation av en generisk list-klass, AList, med en intern array för elementen. Klassen ska metoderna

- size, returnerar antal element,
- get(int index), som returnerar elementet på angiven position; om index är felaktigt ska ett IndexOutOfBoundsException kastas,
- add(E obj), som lägger till ett objekt sist i lista; om interna arrayen är full ska denna automatiskt utökas,
- forEach(ICConsumer<E> consumer), som itererar över elementen i listan och utför den operation som definieras i consumer på varje element.

Interfacet IConsumer ska ha en abstrakt metod, apply(E element), som representerar en operation på ett element. Skriv interfacet och två implementationer av detta enligt nedan.

- Print, som skriver ut elementet.
- MoveAll<IShape>, som förflyttar elementet (som förutsätts vara av typen IShape) ett steg i x- respektive y-led.

Skriv en main-metod där du skapar en lista av typen AList, lägger IShape-objekt i denna lista och anropar metoden forEach för respektive interface-implementation.

Visa sedan att du får samma resultat som ovan genom att anropa forEach med motsvarande lambda-uttryck.

(4 p)